Transitions and scene changes

Why an unrelated update could interrupt an animation, and what changed

A switch can be halfway through moving when something elsewhere in the window changes. Previously, adding or removing an unrelated element could make the switch jump to its final position. The change described here keeps the animation running when its part of the scene can be matched safely.

The visual stress test was tried with unrelated updates both off and on; the user reported no noticeable difference. That is consistent with the intended result: the switch and tiles should continue animating in both modes. This observation is useful, but it is not a timing measurement or a proof that every application will be smooth.

The original problem

A CSS transition gradually changes a property over time. For example:

```
.switch-thumb {
    left: 3px;
    transition: left 0.4s ease;
}
.switch.dark .switch-thumb { left: 27px; }
```

When the class changes, the thumb should move from its current position toward the new one. An unrelated item appearing elsewhere should not, by itself, stop that movement.

The earlier implementation compared the structure of the **entire rendered scene**. If the old and new trees did not have the same shape, it skipped the whole transition. During an existing transition, an unrelated update could also discard the current transition and start a new one. Thus the switch's behavior depended on changes outside the switch.

The day/night example makes this reproducible. Its diagnostic mode adds or removes a node outside the switch. A headless check on the original implementation found an active 400 ms transition with the diagnostic mode off, but no active transition when that structural change happened with the toggle.

Why the older stress test missed it: `gui_effect_pressure` puts pressure on rendering and changes classes in steps. It does not exercise this particular combination of an active CSS transition and repeated, unrelated structural changes.

The targeted change

The transition code now compares scene branches individually instead of rejecting every animation when one branch changes. For a matched branch whose transition target remains the same, the existing animation can continue on its original clock. If its parent moves because of layout, the stored starting position is adjusted so that the animation can continue relative to the moved parent.

Matching is deliberately conservative when a sibling list changes length: elements with unique IDs can be matched, while anonymous or duplicate-ID siblings are not guessed. Equal-length lists retain the existing positional matching behavior. That has an important limitation: without stable identities, the code cannot always distinguish an anonymous element from a replacement or reordered element in the same position.

This is a **targeted improvement**, not a complete browser transition engine. Transitions are still managed through scene snapshots. Different animated properties do not yet have fully independent lifetimes, and some updates can still make the implementation fall back to starting a new scene transition. Browser-specific reversal and layout behavior remain separate work.

How to try it visually

```
cargo run --release --example transition_stress_gui -- 1000
```

The window shows a moving reference switch and a wall of color-transitioning tiles. Compare the animation with **Unrelated changes** off and on. When enabled, a small yellow marker appears and disappears in a separate branch every 300 ms. The controls also let you stop automatic toggling, toggle manually, and add or remove tiles. Some tiles may be clipped by the window, but they still contribute to the scene.

The terminal reports observed frame rate, long frame intervals, and maximum frame-work and paint times every two seconds. These measurements include real desktop rendering, but they can also be affected by the display, operating system, and other applications.

For the smaller reproduction, run:

```
cargo run --release --example daynight
```

Turn its diagnostic mode on, then toggle the switch. Automated tests also cover adding and removing unrelated content during an active transition and repeated toggles.

What the larger synthetic test measured

The headless test creates many color-transitioning elements and repeatedly inserts or removes an unrelated node. It uses simulated 16 ms frame intervals and measures **scene preparation**: it does not paint pixels or present a desktop window. Its counting allocator adds measurement overhead and tracks Rust heap allocations, not all process or GPU memory.

```
cargo run --release --example transition_stress -- 5000 180
```

The table shows medians from five release-build runs on an Apple M1 Pro, with a structural update every five simulated frames. The 95th percentile is a slow-frame indicator: 95% of the measured frames were at or below the stated preparation time.

Structural-change case	Before	After
1,000 elements: 95th-percentile preparation	3.31 ms	4.03 ms
5,000 elements: 95th-percentile preparation	22.73 ms	26.63 ms
5,000 elements: peak tracked Rust heap	106.74 MiB	106.82 MiB
Frames with an active transition, out of 180	6	72

Table 1: The two versions do different amounts of work: the earlier version cancels most transitions when the unrelated structure changes.

At 1,000 elements, the preparation times in this test leave room within a 16.7 ms frame interval, though painting and presentation still need time. At 5,000 elements, the measured slow-frame preparation exceeds that interval in both versions, even before painting. The allocator instrumentation and synthetic workload mean those numbers should not be read as actual desktop frame rates.

The 5,000-element run allocated approximately 4,299 MiB **in total** across 180 measured frames after the change. This is allocation churn, not 4,299 MiB retained at once. The peak tracked live Rust heap was approximately 107 MiB. Both the repeated allocations and the peak are reasons to investigate, rather than proof of a problem in a particular app.

What to check next

Use the visual example and real application traces to decide whether a bottleneck is noticeable. Measure frame intervals, scene rebuilding, transition sampling, painting, and memory separately. If larger realistic scenes need improvement, investigate preserving stable element identity and tracking only active property transitions before adding more scene snapshots or GPU layers. Any larger redesign should be checked against browser behavior and measured rather than assumed faster.

Prompt for an AI coding assistant

Copy this prompt when asking an assistant to work on transitions or rendering. Replace the final line with the code or behavior to investigate.

```
Investigate and update the specified cssimpler code to keep CSS
transitions correct when unrelated UI content changes.

- Reproduce the issue first. Compare the behavior with a browser
  where relevant, and keep a small repeatable regression case.
- Keep an existing animation running on its original timeline when
  its own target has not changed. Do not transfer an animation to a
  different or ambiguous element after an insertion, removal, or
  reorder. Investigate stable identity instead of assuming that an
  array position or a duplicated HTML id is sufficient.
- Keep app state local where practical. Avoid mutable globals used
  merely to connect event handlers to an app instance.
- Distinguish layout-affecting properties from paint-only properties
  and transforms. Preserve correctness for neighbors and clipping;
  do not assume every animation can skip layout or painting.
- Prefer focused changes and existing caches. Avoid whole-scene
  copies, per-frame allocations, or GPU layers unless measurements
  and correctness justify their cost.
- Test unrelated changes both when an animation starts and midway
  through it, repeated toggles, removal and replacement, and changed
  or ambiguous sibling lists. Cover App and FragmentApp when relevant.
- Run the GUI example and inspect the animation as well as its frame
  timing. Benchmark realistic and larger scenes; report tail latency,
  allocation churn and peak memory. State when an older version does
  less work because it cancels animations, and distinguish headless
  preparation from painting and presentation.
- Explain the change, test results, measurements, and any remaining
  browser-compatibility or element-identity limitations.

Files or behavior to investigate: [describe here]
```

There is no single optimization that is best for every scene. Correct matching and realistic measurements should guide the next change.